

Java 9–21 Key Features

Records; sealed classes; text blocks; switch expressions; var keyword; pattern matching

Rahul Guha || SME || Java FS Training || HCL
2. Modern Java (8–21)

Java 9–21 Key Features

Java 9–21 introduced several language and platform improvements that make Java code shorter, safer, more readable, and easier to maintain. The most important features for learning and interviews are var, Text Blocks, Switch Expressions, Records, Sealed Classes, and Pattern Matching.

Note: These features were introduced across different Java versions, not all in one release.

A visual guide to Java 9 through 21 covering JPMS, collection factories, var, HTTP Client, records, sealed classes, pattern matching, switch expressions, text blocks, virtual threads, and sequenced collections.

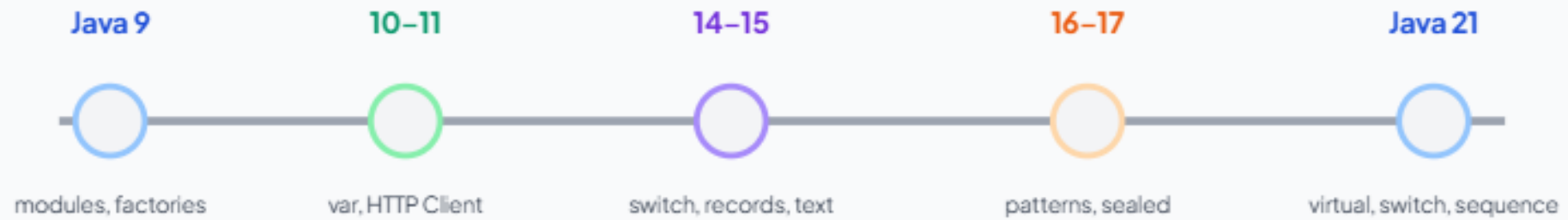
Modern Java did not stop evolving after Java 8. Releases 9 through 21 made the platform more modular, concise, expressive, and scalable while preserving Java's preference for readable, strongly typed code.

What you will learn

- how JPMS modules differ from packages and JARs;
- the contracts of immutable collection factories and var;
- synchronous and asynchronous HTTP Client usage;
- record semantics, defensive copying, and identity trade-offs;
- sealed hierarchy rules and exhaustive pattern matching;
- switch expressions, text blocks, and pattern variables;
- where virtual threads help and where they do not;
- how sequenced collections standardize first/last/reversed access.

1. Release map

Modern Java evolved in layers



Preview features became permanent only after feedback; production code must know the target JDK.

Java's six-month release cadence separates feature arrival from long-term-support adoption.

2. Java Platform Module System (Java 9)

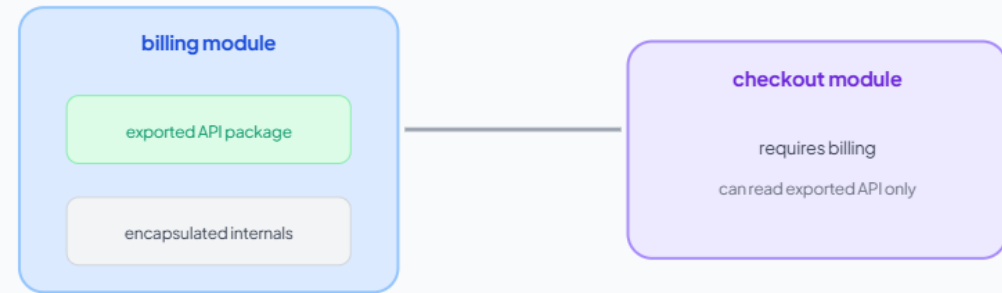
A module is a named set of packages with an explicit dependency and visibility contract.

```
module com.skilledcoder.billing {  
    requires java.sql;  
    requires transitive com.skilledcoder.domain;  
    exports com.skilledcoder.billing.api;  
}
```

module-info.java can declare:

- **requires:** dependency needed to compile/run;
- **requires transitive:** consumers also read that dependency;
- **exports:** packages accessible to other modules;
- **opens:** reflective access, often for frameworks;
- **uses and provides ... with:** service loading.

Modules make architectural boundaries enforceable



A public class in a non-exported package is still inaccessible to ordinary code in another named module.

JPMS adds reliable configuration and strong encapsulation beyond classpath order. The unnamed module preserves classpath compatibility. Automatic modules help migration but expose unstable inferred names and broad exports.

Usually missed: `exports` is not `opens` : Exports allows normal compiled access. Opens permits deep reflection. Framework convenience is not a reason to open every package globally.

3. Collection factory methods (Java 9)

```
List<String> roles = List.of("ADMIN", "USER");  
Set<Integer> ports = Set.of(80, 443);  
Map<String, Integer> scores = Map.of("Ada", 10, "Linus", 9);
```

These collections are unmodifiable, reject null elements/keys/values, and sets/maps reject duplicates. Their implementation type and iteration order are not contracts.

They are not necessarily deeply immutable: mutable contained objects can still change.

List.copyOf, added later, creates an unmodifiable copy and may reuse an already suitable immutable instance.

Do not confuse List.of(array) with primitive arrays: a primitive array becomes one element because it cannot expand as Object[] varargs.

4. Local variable type inference with var (Java 10)

```
var users = repository.findActiveUsers();  
var byId = new HashMap<String, User>();
```

var is compile-time inference, not dynamic typing. The variable still has one exact static type. It works for initialized local variables, enhanced-for variables, and later lambda parameters. It cannot be used for fields, method parameters/returns, an uninitialized local, or a bare null initializer.

Use it when the initializer makes the type obvious; avoid it when the declared type communicates abstraction or numeric precision.

```
var value = 1; // int, not long
```

```
var list = new ArrayList<>(); // inferred ArrayList<Object>
```

`VAR` CAN EXPOSE TOO MUCH IMPLEMENTATION

``var service = new ConcreteService()`` fixes the local type as ``ConcreteService``, not an interface. This can make later reassignment or refactoring less flexible.

5. Standard HTTP Client (Java 11)

```
HttpClient client = HttpClient.newBuilder()
    .connectTimeout(Duration.ofSeconds(3))

    .followRedirects(HttpClient.Redirect.NORMAL)
    .build();
HttpRequest request =
    HttpRequest.newBuilder(URI.create(url))
        .timeout(Duration.ofSeconds(5))
        .header("Accept", "application/json")
        .GET()
        .build();
HttpResponse<String> response =
    client.send(
        request,
        HttpResponse.BodyHandlers.ofString());
```

The client supports HTTP/1.1, HTTP/2, WebSocket, synchronous send, and asynchronous sendAsync returning CompletableFuture.

```
CompletableFuture<User> user = client
    .sendAsync(request, BodyHandlers.ofString())
    .thenApply(HttpResponse::body)
    .thenApply(json::readUser);
```

Reuse clients for connection management.
Always define connect/request timeouts, validate status codes before parsing, bound body size where necessary, and handle interruption/cancellation.

6. Switch expressions (Java 14)

```
int days = switch (month) {  
    case APRIL, JUNE, SEPTEMBER, NOVEMBER -> 30;  
    case FEBRUARY -> leapYear ? 29 : 28;  
    default -> 31;  
};
```

Arrow cases do not fall through. A block uses yield to provide its value:

```
String label = switch (status) {  
    case FAILED -> {  
        audit();  
        yield "Needs attention";  
    }  
    default -> "OK";  
};
```

Expressions must be exhaustive. Enum coverage can omit default, allowing the compiler to detect missing constants after evolution.

7. Text blocks (Java 15)

```
String json = ""  
    {  
        "name": "Ada",  
        "active": true  
    }  
    "";
```

Text blocks improve multiline source readability but do not interpolate values and do not make SQL/HTML safe.

Use parameterized SQL and contextual escaping.

Incidental indentation is removed based on closing delimiter/content indentation. `\` can suppress a line terminator and `\s` preserves a trailing space deliberately.

8. Records (Java 16)

```
public record Money(BigDecimal amount, Currency  
currency) {  
    public Money {  
        Objects.requireNonNull(amount);  
        Objects.requireNonNull(currency);  
        if (amount.signum() < 0) throw new  
IllegalArgumentException("negative");  
    }  
}
```

Records generate private final component fields, accessors, a canonical constructor, equals, hashCode, and toString. They are final and cannot extend another class, but can implement interfaces.

Records are shallowly immutable. A List component can still mutate. Defensively copy in the compact constructor:

Records (Java 16)

```
public record Team(String name, List<Member> members) {  
    public Team { members = List.copyOf(members); }  
}
```

A record declares a transparent state description



9. Pattern matching for instanceof (Java 16)

```
if (value instanceof String text && !text.isBlank()) {  
    System.out.println(text.length());  
}
```

The pattern variable exists only where the match is definitely true. Flow scoping understands boolean control:

```
if (!(value instanceof String text)) return;  
System.out.println(text); // in scope
```

This removes a repeated cast while preserving runtime type checking. It is not structural typing; the tested type remains a nominal Java type.

10. Sealed classes (Java 17) : Direct permitted subclasses must be final, sealed, or non-sealed. This lets the author control the closed set while each subtype chooses whether its branch stays closed. Sealed hierarchies are ideal for domain alternatives, AST nodes, commands, and outcomes. They are not ideal when third parties must freely extend the API.

Java 21 – Virtual Threads

Virtual Threads were finalized as a permanent feature in Java 21 through JEP 444. They are lightweight threads designed to make it easier to build highly concurrent applications, especially applications that perform a large number of I/O-bound tasks such as database calls, HTTP requests, file operations, and network communication. Simple definition: A virtual thread is a lightweight Java thread that is managed by the JVM rather than being directly tied to one operating-system thread for its entire lifetime.

1. Traditional Platform Threads

Before virtual threads, Java applications commonly used platform threads:

```
Thread t = new Thread(() -> {  
    System.out.println("Task running");  
});  
  
t.start();
```

A platform thread is backed by an operating-system thread.

If an application creates a very large number of platform threads, it can consume significant:

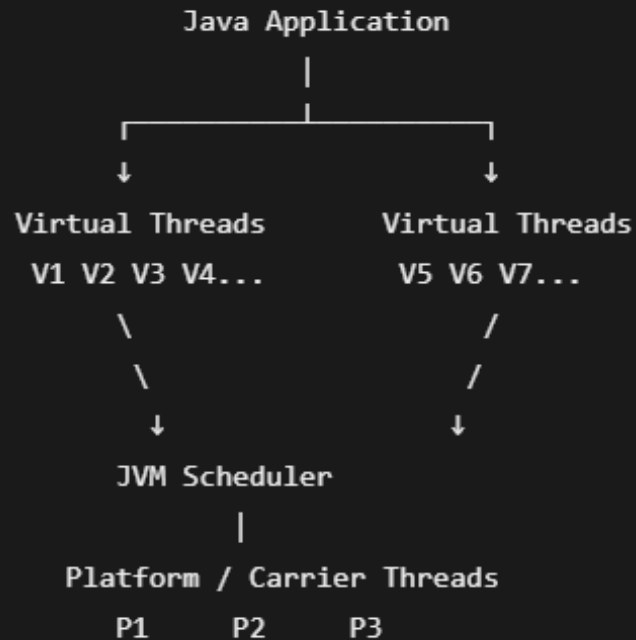
- Memory
- OS resources
- CPU scheduling resources

Java 21 – Virtual Threads

Java 21 allows:

The JVM manages virtual threads and schedules them onto a smaller number of platform threads.

`</>` Plain text



```
public class VirtualThreadDemo {

    public static void main(String[] args) {

        Thread thread = Thread.startVirtualThread(() -> {

            System.out.println("Task running...");

        });
        System.out.println("Main thread running");

    }

}
```